



OBSERVABILITY & OPERATIONS

Book 07 of the series — knowing what your cluster is doing: Probes (the health question Kubernetes can't answer alone), Metrics Server & Prometheus, the exporter cast, log architecture & Loki, Grafana, PromQL/LogQL, events & audit — and the cluster-wide failure playbook.

Probes

liveness · readiness

Metrics

server · top

Prometheus

pull model

Exporters

node · state · cAd

Logs

/var/log/pods

Loki

labels only

Grafana

the pane

Playbook

cluster triage

PART I · PROBES — THE HEALTH QUESTION

- | | |
|--|---|
| ☐ CH 01 Running ≠ Healthy p.3 | ☐ CH 02 Liveness Probes — Restart the Stuck p.4 |
| ☐ CH 03 Readiness — The Traffic Gate p.5 | ☐ CH 04 Startup Probes & Probe Mechanics p.6 |

PART II · METRICS

- | | |
|---|---|
| ☐ CH 05 Metrics Server & kubectl top p.7 | ☐ CH 06 Prometheus — The Pull Model p.8 |
| ☐ CH 07 The Exporter Cast — Who Exposes What p.9 | |

PART III · LOGS

- | | |
|---|--|
| ☐ CH 08 Where Logs Actually Live p.10 | ☐ CH 09 Collection Patterns — DaemonSet · Sidecar p.11 |
| ☐ CH 10 Loki — Prometheus for Logs p.12 | |

PART IV · SEE IT & QUERY IT

- | | |
|---|---|
| ☐ CH 11 Grafana — The Unified Pane p.13 | ☐ CH 12 PromQL & LogQL — First Queries p.14 |
|---|---|

PART V · OPERATIONS

- | | |
|---|---|
| ☐ CH 13 Events & Audit Logging p.15 | ☐ CH 14 The Cluster-Scope Failure Playbook p.16 |
|---|---|

PART VI · LABS

- | | |
|---|--|
| ☐ LAB 1 Probes That Bite p.17 | ☐ LAB 2 Top & Events Triage p.18 |
| ☐ LAB 3 Install the Stack (Helm Preview) p.19 | ☐ LAB 4 Query Everything p.20 |

PART VII · PROVE IT

- | | |
|--|---|
| ☐ QUIZ Self-Check Quiz p.21 | ☐ KEY Answer Key + Scoring p.22 |
| ☐ GLOSS Glossary p.23 | ☐ CHEAT One-Page Cheat Sheet p.24 |
| ☐ FINAL Where to Next → Book 08 p.25 | |



Assumes Books 01–06. Book 01's self-healing restarts **dead** processes — this book's Part I handles the ones that are *alive but broken*. Book 06's HPA already ate from metrics-server; now you meet the whole kitchen. Your DaemonSet agents (Book 06 · CH 01) return as the log collectors.



Book 01 · CH 06 taught self-healing: a Pod dies → reconciliation replaces it. But look closer at what triggers it — **process death**. A deadlocked, memory-leaking, or infinitely-hung application never dies. It just keeps **Running**. Forever. Serving garbage.

WITHOUT PROBES — THE LIE OF THE STATUS COLUMN

```
NAME      STATUS    RESTARTS
api-7d9f  Running  0      # looks perfect...
# reality: deadlocked since Tuesday.
# holds connections open, serves timeouts.
# self-healing will NEVER touch it —
# the process is alive. k8s can't see
# "working" — only "not crashed".
```

WITH PROBES — YOU TEACH K8S WHAT "HEALTHY" MEANS

```
NAME      STATUS    RESTARTS
api-x7k2  Running  37     # restarts = the probe
# KILLING broken containers.
# 37 restarts looks scary —
# it's the system working.
```

The three pillars this book covers (and the fourth)

 **Metrics** how much / how fast

 **Logs** what happened, exactly

 **Probes** is it working NOW

 **Traces** where time went — Book 09's horizon



Probes vs monitoring: probes are **reflexes** — automated, per-Pod, inside the cluster (restart / gate traffic). Metrics and logs are **awareness** — stored, queried, human-facing. You need both; reflexes without awareness is a body that heals but never learns.



One sentence: Kubernetes can only see **life** (the process runs) — probes are how you teach it to see **health** (the process works), and they come in exactly two flavors of consequence: **restart it** (liveness) or **don't send traffic** (readiness) — next two chapters.

 The restart reflex — [CH 02: Liveness Probes](#)

↑ back to top



A liveness probe asks one question on a timer: "are you working?" Fail it enough times and the **kubelet kills the container** — and the restart policy (Book 01) brings it back fresh. Deadlock → cured by machine, not by 3am pager.

```
spec:
  containers:
  - name: api
    image: myapi:2.0
    livenessProbe:
      httpGet:
        path: /healthz      # YOUR endpoint – cheap, local, self-contained
        port: 8080
      initialDelaySeconds: 10 # let the app boot first
      periodSeconds: 10      # ask every 10s
      failureThreshold: 3    # 3 consecutive fails → RESTART
```

FOUR WAYS TO ASK

Mechanism	Healthy when...
httpGet	response 200–399
tcpSocket	TCP connect succeeds
exec	command exits 0
grpc	gRPC health check passes

THE TIMING KNOBS

```
initialDelaySeconds  grace before first check
periodSeconds        ask interval (default 10)
timeoutSeconds       answer deadline (default 1)
failureThreshold     consecutive fails → act
successThreshold     fails needed to heal
(liveness: always 1)
```



The #1 liveness mistake — checking dependencies: if `/healthz` verifies "can I reach the database?", then one DB blip makes **every Pod's liveness fail** → the entire fleet restart-loops *simultaneously* — you converted a database hiccup into a self-inflicted full outage. Liveness = **am I functional?** (process, event loop, local state) — **never** "are my friends up?"

🌱 Probes run constantly — keep them **cheap** : a 2-second liveness check every 10s on 200 Pods is real load

📦 Restarts **crash** (10s, 20s, 40s... capped 5 min) — pathological flapping shows as growing RESTARTS + use the **backoff** CrashLoopBackOff rhythm (Book 01 · CH 23)



One sentence: a liveness probe is a **dead-man's switch for the walking dead** — it lets the kubelet kill what should be dead but isn't, and its cardinal rule is *check yourself, never your dependencies*. Lab 1 pulls the trigger on purpose.

🔗 The gentler sibling — [CH 03: Readiness](#)

↑ back to top



Liveness answers "keep or kill?" — readiness answers a different question entirely: **"send this Pod customers right now?"** A failing readiness probe restarts *nothing*; it just removes the Pod from the Service's endpoints (Book 03 · CH 11 machinery).

```
livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
readinessProbe:
  httpGet: { path: /ready, port: 8080 }
  periodSeconds: 5
  failureThreshold: 2    # pull from rotation fast
  successThreshold: 1    # but return instantly when it recovers
```

WHAT A FAILING READINESS PROBE ACTUALLY DOES



web-1 Ready ✓



web-2 NotReady — gated

▼ Service selector gathers endpoints ▼

```
$ kubectl get endpoints web
NAME      ENDPOINTS
web       10.244.1.5:8080 # web-2 GONE from the list —
                                     # no restarts, no logs, just no traffic
```

	Liveness	Readiness
Question	are you alive, or kill you?	should traffic reach you?
On failure	container RESTARTED	removed from endpoints — nothing else
Recovers by	fresh process	probe passing again — instant re-admission
Use for	deadlocks, hopeless states	warming caches, loading models, overload shedding , DB reconnection



Rolling updates already depended on this: Book 01 · CH 20's "new Pod replaces old only when Ready" — *Ready means the readiness probe passes*. No readiness probe = rolling update slams traffic into Pods that are still booting. This chapter is where that story gets its ending.



The readiness death-spiral check: readiness *may* include dependencies ("can I reach my DB? — if not, shed traffic") where liveness may not. Readiness failing everywhere is **safe** (traffic stops, pods survive, users see errors) — but ensure something restarts or alerts, because a Pod can sit NotReady **forever** and Kubernetes will not lift a finger.



One sentence: readiness is a **traffic valve, not a kill switch** — failing pods drop out of the endpoints list silently and rejoin when ready, which is what makes rolling updates, warm-ups, and overload survival possible.

▶▶ The boot-guard for slow apps — [CH 04: Startup Probes](#)

↑ back to top



Some apps boot slowly — a JVM loading 4GB of heap, a model loading weights. Liveness starts checking at `initialDelaySeconds` ... and if the app needs 90 seconds, you either set a huge delay (slow restarts forever) or watch it **restart-loop during every boot**. The startup probe solves the dilemma: **it disables the other probes until the app is up once**.

```
startupProbe:                # runs FIRST – liveness/readiness wait
  httpGet: { path: /healthz, port: 8080 }
  failureThreshold: 30        # 30 × 10s = up to 5 minutes to boot
  periodSeconds: 10
livenessProbe:                # takes over ONLY after startup succeeds
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 10
  failureThreshold: 3
```

Boot sequence with all three probes:

```
container starts → startupProbe asks every 10s
  | one success (or 30 fails → container killed)
  ▼
  readiness + liveness activate from this moment
  └─ readiness fails → out of endpoints (traffic gate)
  └─ liveness fails → RESTART (and startup runs again first!)
```

Probe	Fires	On failure	One-line job
startup	from container start	restart (after threshold)	guard the boot — protect slow starters from eager liveness
readiness	after startup passes	out of endpoints	gate traffic per moment
liveness	after startup passes	restart container	kill the undead



Rule of thumb: fast-booting app → skip `startupProbe`, use `initialDelaySeconds`. Slow or variable boot → `startupProbe` with generous `failureThreshold × periodSeconds` budget, and tight liveness afterwards. Best of both: instant restart detection *and* no boot-looping.



One sentence: the startup probe is a **one-shot gate that silences the other probes until first success** — letting slow apps boot in peace while keeping restart-reflexes razor-sharp once they're up. All three together = the complete health contract. Lab 1 fires all of them.

Part II: from reflexes to awareness — [CH 05: Metrics Server](#)

↑ back to top



The lightest rung of the metrics ladder: **metrics-server** — a small in-cluster component that scrapes resource usage from every kubelet and serves it through the Metrics API. It's what makes `kubectl top` work and what fed Book 06's HPA.

```
$ minikube addons enable metrics-server
$ kubectl top nodes
NAME          CPU(cores)   CPU%   MEMORY(bytes)  MEMORY%
minikube      347m         17%    1823Mi         47%
$ kubectl top pods -n shop
NAME          CPU(cores)   MEMORY(bytes)
web-x7k2      480m         210Mi # vs its REQUEST (Book 02 · CH 14) — 96% of 500m!
```



Know its ceiling: metrics-server keeps only the **latest ~15 minutes** of samples. No history, no graphs, no alerts, no dashboards. It answers "how much is it using *right now*?" — long-term awareness is Prometheus's job (CH 06). `top` numbers also lag reality by ~1 minute (scrape + aggregation).

Its two customers: `kubectl top` (you) and the **HPA's Resource metrics** (Book 06 · CH 07) — no metrics-server, no autoscaling on cpu/memory

`top` shows **actual usage**; compare `requests` to spot right-sizing candidates — that comparison is exactly what VPA automates (Book 06 · CH 08)



One sentence: metrics-server is the **short-term pulse** — live CPU/memory per node and Pod for `top` and the HPA, deliberately keeping nothing older than minutes; everything beyond that is Prometheus's territory, next chapter.

The real memory — [CH 06: Prometheus](#)

↑ back to top



Prometheus is the de-facto metrics database of the Kubernetes world. Its defining choice: it **pulls** — it scrapes each target's `/metrics` HTTP endpoint on an interval, storing every measurement as a **time series**.

```
# a target's /metrics endpoint is just text – self-describing:
$ curl localhost:8080/metrics
http_requests_total{method="POST",code="200"} 1027
http_request_duration_seconds{quantile="0.99"} 0.14
# metric NAME + LABELS (dimensions) + current VALUE –
# scraped over time, each combo becomes a time series
```

Time Series

name + labels + timestamped values — the only data shape. Labels are the dimensions you'll slice by.

PromQL

query language built for rates, aggregation, and math over series (CH 12).

Alerting

PromQL conditions → **Alertmanager** → Slack/PagerDuty/email routing.



Why pull wins here: the scraper controls the pace and the target list; apps just expose `/metrics` and need zero knowledge of where Prometheus lives. Bonus: a dead target doesn't vanish — its `up` metric becomes `0`, so you can query whether the monitoring itself sees the target: `up == 0` is the oldest trick in the Prometheus book.

IN A CLUSTER — WHO WATCHES THE WATCHERS



target discovery is automatic — via Service discovery + **ServiceMonitor/PodMonitor CRDs** (next chapter), never hand-maintained IP lists



One sentence: Prometheus = **pull-based time-series machine** — it scrapes self-describing `/metrics` endpoints, keeps every label-combination as its own series, answers with PromQL, and even reports its own blindness via `up == 0`.

Where all those endpoints come from — [CH 07: The Exporter Cast](#)

↑ back to top



Prometheus only scrapes — someone must **expose** the numbers. Three built-in exporters cover the whole cluster, each answering a different "is it healthy?" question:

Exporter	Runs as	Exposes	Answers
node-exporter	DaemonSet (Book 06 · CH 01!)	host-level: CPU, memory, disk, network of the machine	"is the <i>machine</i> healthy?"
cAdvisor	embedded in the kubelet	per-container actual usage: CPU, memory, restarts	"which <i>container</i> eats resources?"
kube-state-metrics	Deployment, watches the API Server	object states: replicas desired vs ready, pending pods, PVC capacity	"does <i>Kubernetes</i> think all is well?"



The layer trick: the same Pod can be healthy in all three views at once — or exactly one. High node CPU (node-exporter) + low pod CPU (cAdvisor) = something else on the machine. Pod Running + `kube_pod_status_phase{phase="Pending"}` climbing (kube-state-metrics) = scheduling trouble (Book 04!). Three lenses, one truth.

ServiceMonitor — declarative scrape config

```
apiVersion: monitoring.coreos.com/v1      # a CRD — Book 08 previews CRDs
kind: ServiceMonitor
metadata: { name: web, labels: { release: mon } }
spec:
  selector: { matchLabels: { app: web } }
  endpoints: [{ port: metrics }]
# "scrape any Service labeled app=web at port `metrics`" —
# deploy your app WITH its ServiceMonitor → deploy = monitored (Book 02 · CH 04b pattern)
```

```
$ helm install mon prometheus-community/kube-prometheus-stack
# one chart = Prometheus + Alertmanager + Grafana +
# node-exporter (DS) + kube-state-metrics + the CRDs — the full bundle
```



One sentence: cluster metrics come from a three-layer cast — **node-exporter (machines, per node)**, **cAdvisor (containers, via kubelet)**, **kube-state-metrics (object states, via API)** — and your own apps join by exposing `/metrics` plus a ServiceMonitor CRD.

▶ Part III: the other signal — [CH 08: Where Logs Live](#)

↑ back to top



Every container's **stdout** and **stderr** is captured by the container runtime and written to a **plain file on the node**. That's the entire native logging story — and its limits define everything else in Part III.

```
$ kubectl logs web-x7k2      # the API reads node files FOR you...
$ minikube ssh
$ ls /var/log/pods/
default_web-x7k2_e1a2.../api/0.log # ns_pod_UID/container/N.log
# (N = restart count --previous reads N-1! Book 01 · CH 24)
$ ls /var/log/containers/
web-x7k2_default_api-e1a2...log # symlinks – stable names for collectors
```

Fact	Consequence
Logs are node-local files	<code>kubectl logs</code> must ask the kubelet of <i>that</i> node — Pod gone from node = logs gone
Runtime rotates them (containerd: 10Mi × 5 default)	chatty apps wrap around; old logs vanish silently
Survives container restarts (per-Pod dir), dies with the Pod	Book 05's three-lives ladder, log edition
Survives nothing else — no cluster-wide store exists	node loss / eviction (Book 04 drain!) = history gone — hence collectors (CH 09)



The logging manifesto, 12-factor style: your app writes **unbuffered to stdout/stderr** — no log files inside the container, no rotation logic. The platform (runtime + collector) handles transport and storage. Apps that insist on writing `/var/log/app.log` inside the container force you into sidecar workarounds (CH 09).



The eviction surprise: you debug an incident, run `kubectl delete pod` to "reset" it — and the evidence dies with the Pod dir. **Grab logs before deleting:** `kubectl logs NAME > incident.log`, or better, have a collector already shipping (CH 09–10) so the question never arises.



One sentence: container logs are **rotating text files on the node** — `kubectl logs` is just a window onto them, and everything you don't ship off-node (CH 09) dies with the Pod, the rotation, or the node itself.

Shipping them somewhere safe — [CH 09: Collection Patterns](#)

↑ back to top

Node files don't walk to your database by themselves. Two production patterns move them — and you already know the machinery of both.

PATTERN 1 · NODE-LEVEL AGENT (DAEMONSET)
★ **DEFAULT**

```
every node:
  🟢 agent (Promtail / Fluent Bit)
    |— reads /var/log/pods/** (hostPath mount)
    |— attaches metadata: ns, pod, container, node
    |— ships → Loki / ES / cloud
one agent covers EVERY pod on the node –
zero per-app config, app stays ignorant
```

PATTERN 2 · SIDECAR STREAMER

```
one pod:
  🟢 app —writes→ emptyDir volume (Book 05 · CH 02!)
  🟢 sidecar —tails file→ ships to backend
app CAN'T use stdout? per-app config, filtering, multiple destinations?
the sidecar is app-owned plumbing
```

	 DaemonSet agent	 Sidecar
Setup	once per cluster	every Pod that needs it
Resource cost	one agent per node (shared)	multiplied per Pod
Coverage	everything on stdout/stderr — automatic	only Pods you modified
Special needs	none possible per-app	file-based logs, per-app parsing, routing
Verdict	start here — 90% of clusters	escape hatch for stubborn apps

 **Pattern 3 exists and is usually a trap:** the app pushes directly to the logging backend (embedded client, async handler). Rejected because: credentials in every app (Book 02 lesson), retry/buffer logic duplicated everywhere, and backend outage now *breaks your app's logging path* — sometimes its threads. Centralize transport; keep apps dumb.

 **One sentence:** ship logs with a **DaemonSet agent reading /var/log/pods** — one deployment, zero app changes — and reach for the **sidecar** only when an app refuses to write stdout; never embed shipping clients in the app itself.

 Where the shipped logs go — [CH 10: Loki](#)

[↑ back to top](#)



Loki (Grafana Labs) is log aggregation with one radical idea: **index the labels, never the text**. Full-text engines (ELK) index every word — powerful, expensive. Loki asks: *"why index what you never query?"*

💰 THE TRADE

10-20× cheaper storage — only metadata is indexed

Log lines stored **compressed as chunks** in cheap object storage

Search = **labels first**, then grep the few surviving lines at query time

Trade-off accepted: free-text search across everything is weaker than ELK — in Kubernetes, labels are so good it rarely matters

🏷️ THE SHARED LABEL MODEL

Promtail labels a line with:
namespace · pod · container · node
 — the SAME labels your metrics carry
 (Book 03 · CH 11's service DNS model) —
 CPU spike on pod X in Grafana →
 jump to logs of pod X — zero config

Promtail's four-step pipeline (your DaemonSet agent from CH 09, Loki edition)

- 1 **Discover** via k8s Service Discovery — every pod on my node
- 2 **Tail** /var/log/pods files (CH 08!)
- 3 **Label** attach ns/pod/container/node metadata
- 4 **Ship** HTTP PUSH batches → Loki's Distributor

inside Loki: Distributor → Ingester (chunks) → Store → Querier
 — a Log Stream = one unique label combination



Cardinality is the cardinal sin: every unique label combo = a new stream to index. `user_id`, `request_id` as labels → millions of streams → Loki drowns. Labels for **bounded, structured** dimensions (ns, pod, app, level) — everything variable goes *in* the line, filtered at query time.



Push vs pull, the asymmetry: Prometheus **pulls** metrics (CH 06); Promtail **pushes** logs (files won't offer themselves). Two transport models, one shared label vocabulary — that pairing is the stack's superpower.



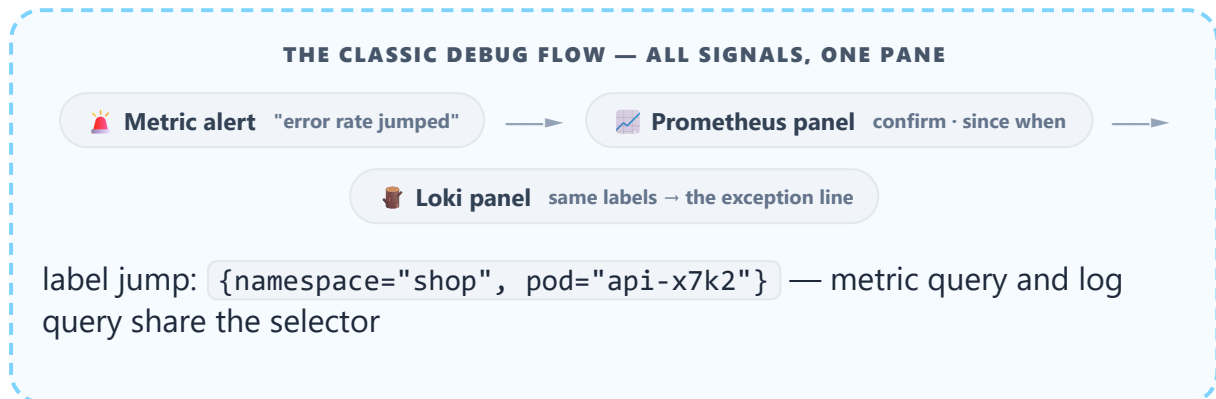
One sentence: Loki is "**Prometheus for logs**" — index only bounded labels, keep raw lines in cheap chunks, let Promtail's DaemonSet do the labeling — and the same label model makes logs and metrics correlatable out of the box.

▶ The window onto all of it — [CH 11: Grafana](#)

↑ back to top



Grafana stores **nothing**. It's a stateless query-and-render layer over your data sources — the glass pane in front of Prometheus (metrics) and Loki (logs). Empty datasources = empty dashboard; the data always lives in the backends.



🔧 **Datasources** — Prometheus, Loki, and dozens more; kube-prometheus-stack wires them in at install (Lab 3)

📊 **Dashboards** — panels of PromQL/LogQL; community dashboards (node full-coverage, k8s cluster views) import by ID in seconds

📄 **Dashboards as code** — export JSON, version in Git next to your manifests — observability joining your declarative world

🔔 **Alerting UI** — Grafana can evaluate alert rules too; deep setups keep alerting in Prometheus/Alertmanager (CH 06) and use Grafana for visualization + triage



The datasources-conflict trap: kube-prometheus-stack deploys Grafana *itself* (it's part of the bundle) and auto-configures its Prometheus datasource. If you later install Grafana separately and wonder why panels are empty — you're looking at **a different Grafana with no datasources**. One pane per cluster (or deliberately many, each configured).



One sentence: Grafana is the **stateless window** — metrics and logs stay in Prometheus/Loki, and its value is the shared-label jump from "the graph looks wrong" to "here's the exact exception line" in seconds.

🔗 Speaking both languages — [CH 12: PromQL & LogQL](#)

↑ back to top



Both query languages share DNA (that's deliberate — Loki's LogQL is *designed* to feel like PromQL). Learn the four moves and you can interrogate both signals.



PromQL — four moves

```
# 1 · SELECT — by name + label matchers
container_cpu_usage_seconds_total{namespace="shop"}

# 2 · RATE — counters need a rate over a window
rate(http_requests_total[5m])

# 3 · AGGREGATE — sum/avg by label dimension
sum by (pod) (rate(container_cpu_usage_seconds_total
  {namespace="shop", container!=""} [5m]))

# 4 · CONDITION — the alert-shaped question
sum by (pod) (rate(http_requests_total{code=~"5.."}[5m]))
/ sum by (pod) (rate(http_requests_total[5m])) > 0.05
```



LogQL — same shape, logs

```
# selector (LABELS ONLY — CH 10) then line filters then parse:
{namespace="shop", pod=~"web-.*"}
  |= "ERROR"           # line contains
  != "health"          # exclude noise

# metrics FROM logs — count matches per minute:
sum by (pod) (rate({namespace="shop"} |= "Timeout" [5m]))

# and the correlation close (CH 11):
# panel 1: rate(http_requests_total{pod=~"web-.*"}[5m])
# panel 2: {pod=~"web-.*"} |= "ERROR" ← same pod regex!
```



Counter law: counters only go up (restarts reset them) — **always** wrap them in `rate()` over a window; the raw value is meaningless across restarts. Gauges (memory, queue depth) you query raw. This one rule separates useful dashboards from lying ones.



One sentence: both languages are **select** → **(rate)** → **aggregate** → **condition** — learn it once on metrics, reuse the shape on logs, and let shared labels stitch the two into one investigation. Lab 4 runs these live.

▶ Part V: the operational paper trail — [CH 13: Events & Audit](#)

↑ back to top



Two different diaries. **Events**: what the controllers did to your objects (you've read them in every `describe` since Book 01). **Audit log**: what *humans and clients* did to the API — the who-did-what record.

EVENTS — THE CONTROLLER CHRONICLE

```
$ kubectl get events --sort-by=.lastTimestamp
LAST SEEN   TYPE      REASON      OBJECT
12s         Warning   BackOff     pod/web-x7k2
4m          Normal    Scheduled   pod/web-x7k2
# Normal = information · Warning = attention
# TTL ~1 hour — they EXPIRE, not an archive.
# per-namespace scope (Book 01 scoping!)
```

Every debugging chapter used them — now you know their lifecycle: generated by controllers/kubelet, aggregated per object, **ephemeral**. For incident forensics, ship them (EventExporter → Loki) or lose them.

AUDIT LOG — THE API DASHCAM

```
# apiserver feature — OFF by default.
# you supply a policy (stages):
who:    user / serviceaccount (Book 02!)
what:   verb + resource       delete secrets...
when:   timestamp + source IP
result: allowed / denied     (RBAC verdicts)
# policy levels: None → Metadata →
# Request → RequestResponse (noisy!)
```

The record that answers "who deleted the production namespace?" — RBAC says who *may*; audit says who *did*. Compliance surveys live here. Concept-level for us; enablement is cluster-lifecycle work (Book 09).



Triage pairing: something's wrong → `kubectl get events --sort-by=.lastTimestamp` (what did the *system* do) + audit log (what did the *humans* do right before). Between the two diaries, almost every "nobody changed anything" mystery resolves — *somebody* did, and the dashcam has the plate number.



One sentence: events are the **system's own short-term diary** (1-hour TTL — read them fast), audit logging the **opt-in API dashcam** recording every identity, verb, and verdict — together they cover both "what happened to my workload" and "who touched my cluster."

 Everything assembled — [CH 14: The Failure Playbook](#)

↑ back to top



Books 01–06 debugged *one Pod*. Real incidents are cluster-shaped: whole namespaces Pending, every DNS lookup dying, a node evacuating itself. The playbook — first move for each signature (every row is a callback):

Signature	Probable cause	First move
Everything in one ns → Pending	ResourceQuota exhausted (B02) or no node fits (B04 funnel)	<code>kubectl describe quota -n ns</code> · read a Pod's events
All pods ImagePullBackOff, one ns	imagePullSecrets missing/wrong (B02 · CH 06)	<code>kubectl get sa default -o yaml</code> → check secret
One node NotReady, pods evacuating	kubelet/heartbeat lost → NoExecute taints (B04 · CH 05)	<code>kubectl describe node N</code> → conditions + taints
ALL DNS failing cluster-wide	CoreDNS pods down (B03 · CH 11–12)	<code>kubectl get pods -n kube-system -l k8s-app=kube-dns</code>
Ingress: everything 404/timeout	controller Pod down, or rulebook orphaned (B03 · CH 04)	<code>kubectl get pods -n ingress-nginx</code> · <code>get ingress</code> → ADDRESS?
Random OOMKills across teams	node memory pressure → eviction by QoS (B02 · CH 15)	<code>kubectl top nodes</code> · check node <code>memory-pressure</code> taint
API slow/5xx for everyone	etcd struggling / quorum at risk (B01 · CH 08)	check static-pod etcd on CP nodes (B04 · CH 11) · <code>events</code>
PVs stuck Pending cluster-wide	provisioner/sc issues (B05 · CH 04/07)	<code>kubectl get sc</code> → default present? · check CSI pods

The **universal triage reflex** (before any of the rows above):

- 1 · `kubectl get nodes` is the fleet itself healthy?
 - 2 · `kubectl get pods -A` which scope? one ns / one node / everywhere?
 - 3 · `kubectl get events -A --sort-by=.lastTimestamp` the chronicle (CH 13)
- scope FIRST — "one namespace" and "one node" are different books of the same incident.



Scope is the diagnosis: one Pod → Book 01's toolbox. One namespace → quota/secrets/RBAC. One node → taints/kubelet/disk. Everything → control plane, DNS, or the network fabric. Nine books of knowledge, organized by blast radius.



One sentence: cluster incidents resolve by **scope-first triage** — nodes, then pods -A, then events — and the playbook above turns each signature into the exact chapter of this series that owns it. Lab 2 drills the reflex.

▶ Prove all of it with your hands — [Part VI: Labs](#)

↑ back to top

**STEP 1 · A pod whose health is a file — liveness**

```
$ kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata: { name: probe-lab }
spec:
  containers:
  - name: c
    image: busybox:1.36
    command: [sh,-c,'touch /tmp/healthy; sleep 9999']
    livenessProbe:
      exec: { command: [cat, /tmp/healthy] }
      periodSeconds: 5
      failureThreshold: 2
EOF
$ kubectl get pod probe-lab # Running, READY 1/1, 0 restarts
```

STEP 2 · Break it — the kubelet fights back

```
$ kubectl exec probe-lab -- rm /tmp/healthy
$ kubectl get pod probe-lab -w
probe-lab Running 0
probe-lab Running 1 # 2 fails → KILLED & restarted (fresh file! CH 02)
$ kubectl describe pod probe-lab | grep -A2 Events | tail -2
Container c failed liveness probe, will be restarted
```

00 READINESS — THE GATE

```
$ kubectl patch pod probe-lab \
  --type=merge -p '{"metadata":{"labels":{"app":"g"}}}'
$ kubectl expose pod probe-lab \
  --port=80 --name=gate
$ kubectl get endpoints gate
gate 10.244.0.5:80
# remove the health file again →
# readiness (same probe) fails →
$ kubectl get endpoints gate
gate <none> # GATED — pod still
# Running, zero restarts!
# (readiness ≠ liveness, CH 03)
```

01 READ THE RESTART STORY

```
$ kubectl get pod probe-lab
READY RESTARTS
1/1 2 # each restart =
# the reflex working.
# RESTARTS ≠ broken —
# it's the cure being
# applied (CH 01!)
```

Proved: liveness kills the undead; readiness gates traffic without touching the process. **Teardown:** `kubectl delete pod probe-lab svc/gate`.

↑ back to top



STEP 1 · The pulse — metrics-server

```
$ minikube addons enable metrics-server
$ kubectl top nodes # give it ~60s after enabling
NAME          CPU(cores)   CPU%   MEMORY(bytes)  MEMORY%
minikube      350m         17%    1800Mi         46%
$ kubectl top pods -A --sort-by=memory | head -5
# the whole cluster, ranked by appetite –
# your right-sizing shortlist (Book 06 · CH 08)
```

STEP 2 · Cause a failure, then read the chronicle

```
$ kubectl create deploy ghost --image=nginx:1.999
$ kubectl get events --sort-by=.lastTimestamp | grep -i ghost
Warning Failed Failed to pull image "nginx:1.999"
Warning BackOff Back-off pulling image
# the incident wrote its own report –
# BEFORE you ran a single describe (CH 13)
```

STEP 3 · The scope-first reflex (CH 14)

```
$ kubectl get nodes # 1 · fleet healthy?
$ kubectl get pods -A | grep -v Running # 2 · blast radius?
$ kubectl get events -A --sort-by=.lastTimestamp | head -8 # 3 · chronicle
# three commands, scope identified, playbook row selected –
# the CH 14 reflex, now muscle memory
```

Proved: live usage via top, self-writing incident chronicles via events, and the universal triage reflex. **Teardown:** `kubectl delete deploy ghost`.

[↑ back to top](#)



Honesty first: the stack ships as **Helm charts** — Book 08's subject. Today you run four commands as a *preview*; Book 08 explains what every flag did. Needs ~3GB free in minikube: `minikube start --memory=6144` if tight.

STEP 1 • Metrics + logs + pane — four commands

```
$ curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
$ helm repo add grafana https://grafana.github.io/helm-charts && helm repo update
$ helm install mon prometheus-community/kube-prometheus-stack
$ helm install loki grafana/loki-stack \
  --set promtail.enabled=true # ← the CH 09 DaemonSet agent!
$
```